Machine Learning Fundamentals in Python
-
Model Evaluation

Martin Řeřábek

ZHAW

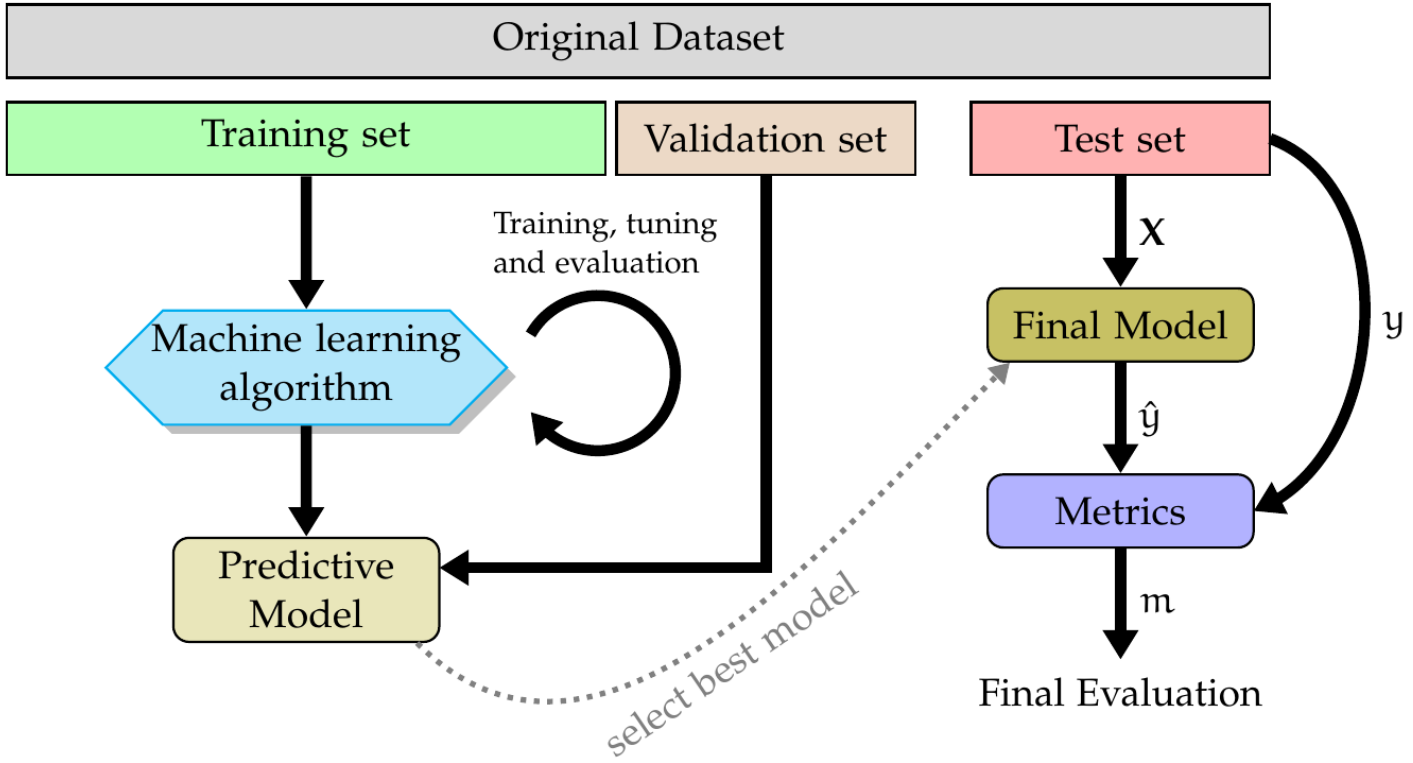
rera@zhaw.ch

Week 4

Overview

1. Data Split
2. Model Evaluation - Metrics
3. Model Evaluation - Cross-Validation
4. Model Diagnostic
5. Model Tuning

Data Split



Training set	Validation set	Testing set
- Model is trained - Usually 80% of the dataset	- Model is assessed - Usually 20% of the dataset - Also called hold-out or development set	- Model gives predictions - Unseen data

Classification Model Evaluation I

- Confusion matrix
  - Popular way to evaluate a classification model - not a metric
  - Created by comparing the predicted class label of a data point with its actual class label
  - Multiclass - series of binary classification problems using ONE vs REST approach

Classification Model Evaluation II

		PREDICTED LABELS	
		n' (Predicted)	p' (Predicted)
TRUE LABELS	n (True)	**True Negative** (Number of instances of negative class 'n' correctly predicted)	**False Positive** *(Number of instances of negative class 'n' incorrectly predicted as the positive class 'p')*
	p (True)	**False Negative** *(Number of instances of positive class 'p' incorrectly predicted as the negative class 'n')*	**True Positive** *(Number of instances of positive class 'p' correctly predicted)*

Classification Model Evaluation III

- True Positive (TP)
  - Total number of instances from the positive class where we correctly predicted the positive class label with our model
- True Negative (TN)
  - Total number of instances from the negative class where we correctly predicted the negative class label with our model
- False Positive (FP)
  - Total number of instances from the negative class where our model misclassified them by predicting them as positive
- False Negative (FN)
  - Total number of instances from the positive class where our model misclassified them by predicting them as negative

Classification Performance Metrics I

- Classification report in Python => Accuracy, Precision, Recall, f1-score, support
- Accuracy
  - $Accuracy = \frac{TP+TN}{TP+FP+TN+FN}$
  - Percentage of predictions that were made correctly
  - Use when classes are almost balanced and correct predictions of those classes are equally important

Classification Performance Metrics II

- Precision
  - What proportion of predicted positives are truly positive?
  - $Precision = \frac{TP}{TP+FP}$
  - Positive predictive value
  - Number of predictions made that are actually correct/relevant out of all the predictions based on the positive class
  - Model with high precision will identify a higher fraction of positive class as compared to a model with a lower precision
  - Precision becomes important in cases where we are more concerned about finding the maximum number of positive class even if the total accuracy reduces
  - Optimizing model for precision - in case that we want to decrease the number of false positives

Classification Performance Metrics III

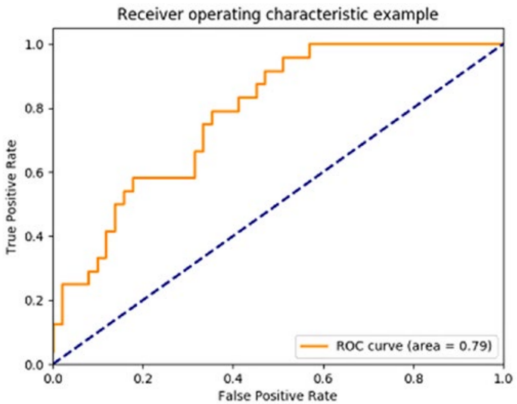
- Recall - sensitivity/hit rate
  - What proportion of actual positives are correctly classified?
  - $Recall = \frac{TP}{TP+FN}$
  - important measure of classifier performance in scenarios where we want to catch the most number of instances of a particular class even when it increases our false positives
  - Optimizing model for recall - in case that we want to decrease the number of false negative
- F1 Score
  - Precision and recall - trade-off relationship
  - Harmonic mean of precision and recall
  - $F1Score = \frac{2 \times Precision \times Recall}{Precision + Recall}$
  - Optimize a classifier for balanced precision and recall performance

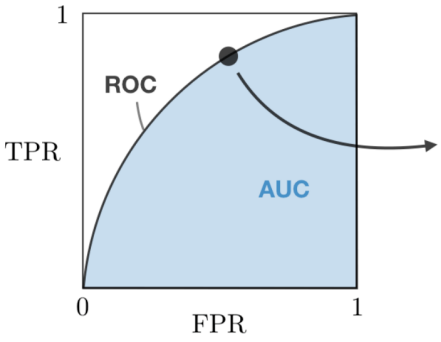
Performance Metrics - Overview

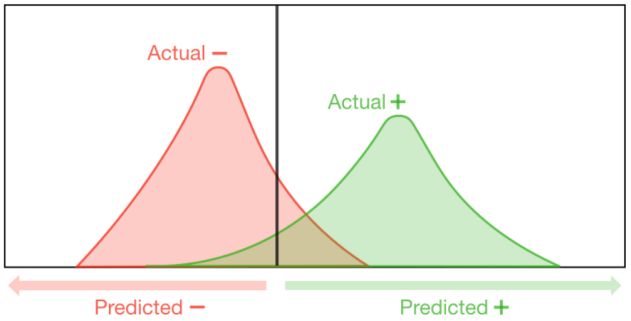
Metric	**Formula**	**Interpretation**
Accuracy	$\frac{TP + TN}{TP + TN + FP + FN}$	Overall performance of model
Precision	$\frac{TP}{TP + FP}$	How accurate the positive predictions are
Recall Sensitivity	$\frac{TP}{TP + FN}$	Coverage of actual positive sample
Specificity	$\frac{TN}{TN + FP}$	Coverage of actual negative sample
F1 score	$\frac{2TP}{2TP + FP + FN}$	Hybrid metric useful for unbalanced classes

Receiver Operating Characteristic Curve

- ROC - considers all possible thresholds for a given classifier and shows the false positive rate (FPR) against the true positive rate (TPR)
- TPR = recall
- $FPR = \frac{FP}{FP+TN}$ - fraction of false positives out of all negative samples
- Metric - AUC







Evaluating Regression Models

- Coefficient of Determination R^2
  - Measures the proportion of variance in y which is explained by X
  - 1 denotes a perfect regression model indicating that all of the variance in dependent variable y is explained by the independent variables X
  - $R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$
    - sum of squares of residuals: $SS_{\text{res}} = \sum_i (y_i - \hat{y}_i)^2$
    - total sum of squares $\propto$ variance of the data: $SS_{\text{tot}} = \sum_i (y_i - \bar{y})^2$
  - Perfect prediction results in $SS_{\text{res}} = 0$ and $R^2 = 1$
  - Baseline model - always predicts mean $\bar{y}$ - will have $R^2 = 0$
  - Worse model - $R^2 < 0$
- Mean Squared Error - calculates the average of the squares of the errors or deviation between the actual value and the predicted values, as predicted by a regression model

Evaluating Clustering Models I

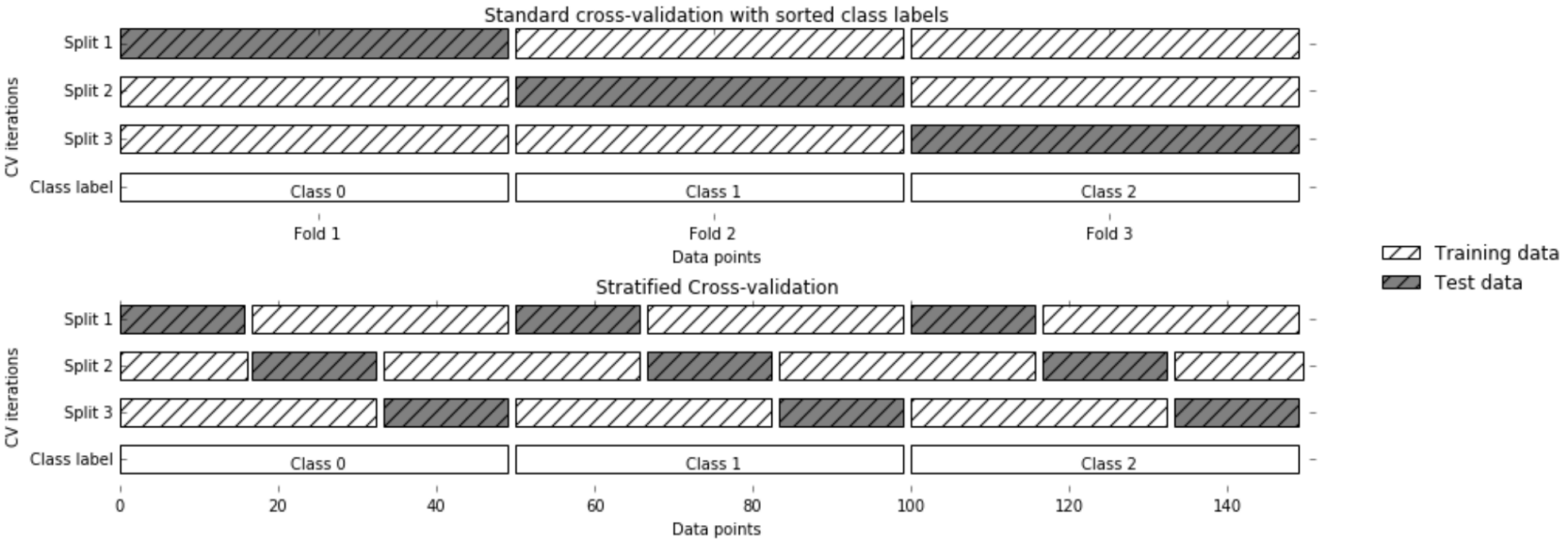
- Complex - annotations not available
- External validation - if some labeled data avialable
  - Homogeneity: A clustering model prediction result satisfies homogeneity if all of its clusters contain only data points that are members of a single class (based on the true class labels).
  - Completeness: A clustering model prediction result satisfies completeness if all the data points of a specific ground truth class label are also elements of the same cluster.
  - V-measure: The harmonic mean of homogeneity and completeness scores gives us the V-measure value.
  - Values are typically bounded between 0 and 1 and usually higher values are better.
  - sklearn.metrics.homogeneity_completeness_v_measure

Evaluating Clustering Models II

- Internal Validation - defining metrics that capture the expected behavior of a good clustering model
  - A good clustering model can be identified by two very desirable characteristics:
    - Compact groups, i.e. the data points in one cluster occur close to each other
    - Well separated groups, i.e. two groups/clusters have as large distance among them as possible.
  - Silhouette Coefficient - sklearn.metrics.silhouette_score
  - Calinski-Harabaz Index - sklearn.metrics.calinski_harabaz_score

Cross-Validation

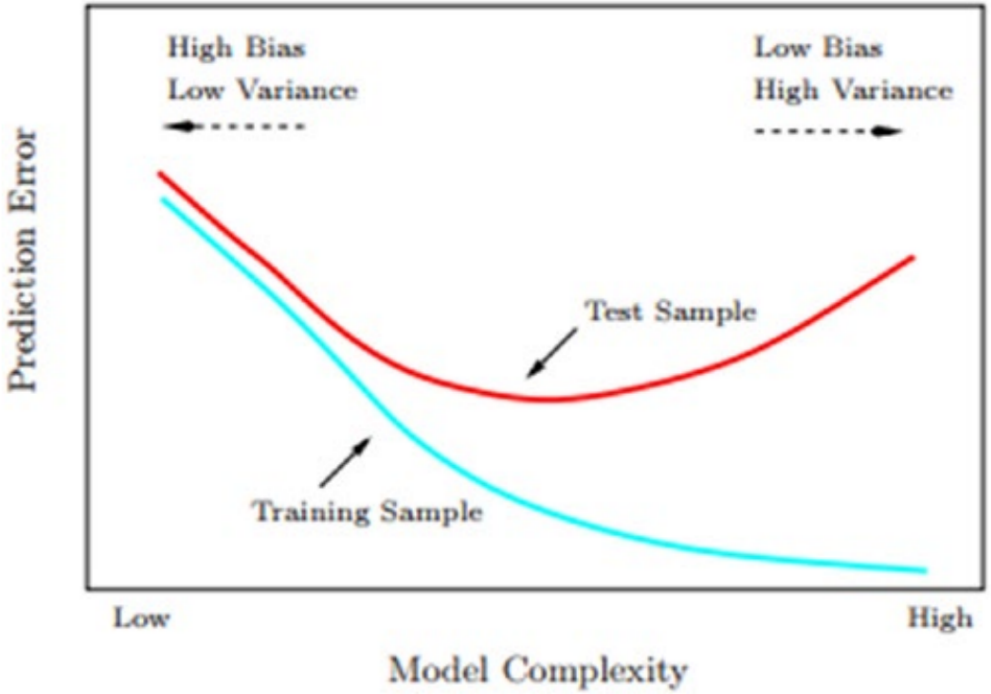
- statistical method of evaluating generalization performance that is more stable and thorough than using a split into a training and a test set
- purpose of cross-validation is to evaluate how well a given algorithm will generalize when trained on a specific dataset

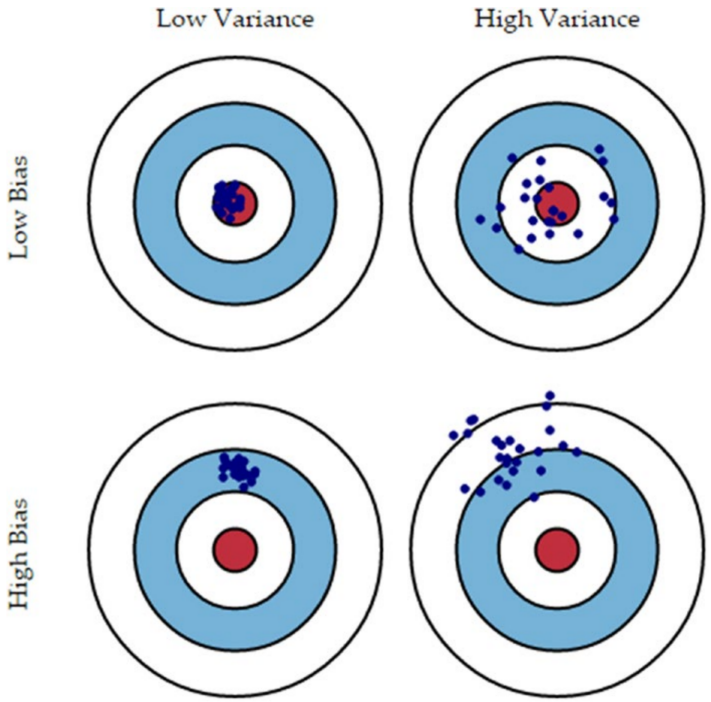


Bias-Variance Trade-off I

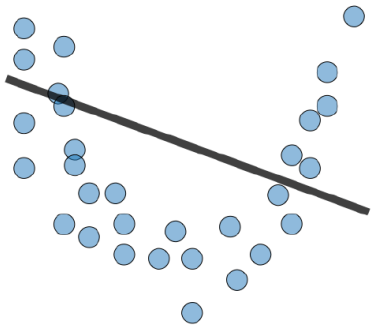
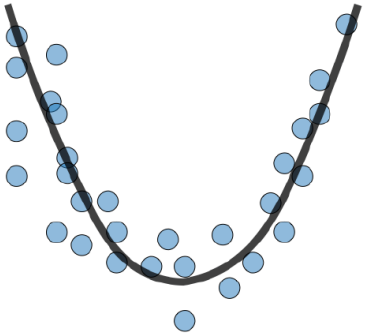
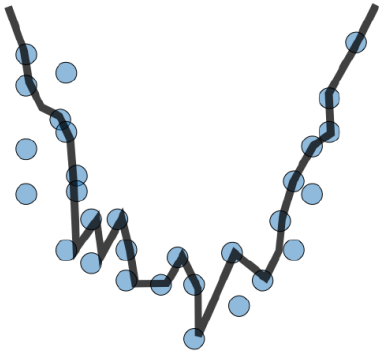
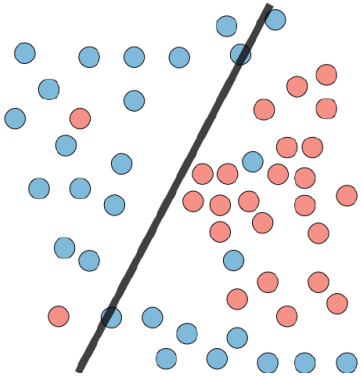
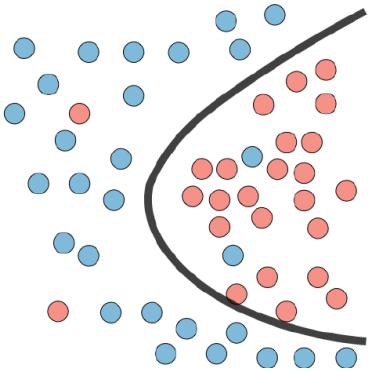
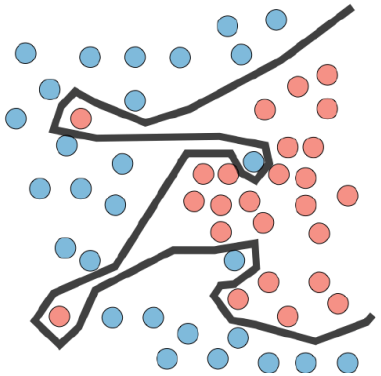
- Bias - the difference between the expected prediction and the correct value that we try to predict for given data points
- Variance - The variance of a model is the variability of the model prediction for given data points
- Trade-off
  - Generalization Error = Bias Error + Variance Error + Irreducible Error
  - Focus on first two errors
  - Every model needs to make a tradeoff between the two choices:
    - making assumptions about the structure of data - underfitting
    - fitting itself too closely to the data at hand - overfitting

Bias-Variance Trade-off II





Bias-Variance Trade-off III

Underfitting	**Just right**	**Overfitting**
- High training error - Training error close to test error - High bias	- Training error slightly lower than test error	- Low training error - Training error much lower than test error - High variance
		
		
- Complexify model - Add more features - Train longer		- Regularize - Get more data

Hyperparmeters

- What are hyperparameters?
  - Meta parameters that are associated with any Machine Learning algorithm and are usually set before the model training and building process
  - Hyperparameters do not have any dependency on being derived from the underlying dataset on which a model is trained = not dependent on data
  - specific for each algorithm and its implementation - check syntax!!!
- Systemic vs Random approach
- Scikit-learn library has an excellent built-in support for performing hyperparameter search with cross-validation
- In case of many parameters - too many combinations to try with grid search - apply first random and then fine tune using grid with smaller steps

Regularization

- Linear regression works by selecting coefficients for each independent variable that minimizes a loss function.
- If the coefficients are too large, it can lead to model over-fitting on the training dataset
- Such a model will not generalize well on the unseen data
- To overcome this shortcoming, we do regularization which penalizes large coefficients.
  - Lasso
  - Ridge
  - Elastic Net